

鳩ナビ おつかいクエスト — 審査員向け 技術想定Q&A

本資料について ハッカソン審査での**技術質問に即答**するための想定問答集です。すべて実装コードに基づき、**2体のエージェント（作成役・検証役）で相互チェック**済み。参照は原則 **ファイル名＋関数/定数名**で示します（行番号は編集でずれるため補助扱い）。

★ 即答の大原則（まずこれだけ覚える）

1. **AIの仕事は2つだけ** … 「**巡回順の提案**」と「**食育クイズ生成**」。方角計算・現在地推定・進行制御・危険判定・スキャン・会計は**すべて端末内のローカル処理**。
2. **禁止ワードを言わない** … 「最短/最適ルート」「AIルート生成」は使わない。正しくは「**後戻りの少ない巡回順**」。会計連携先サービスは**将来構想・未接続**。
3. **誤クイズは絶対に出さない** … 事実グラウンディング→生成抑制→多段検証→固定クイズへフォールバックの**4層防御**。
4. **何があっても止まらない** … AI 8秒タイムアウト、センサー無しでも進行、カメラ無しでもスキップ、音声非対応でも無音続行。
5. **限界は正直に** … 方角は「近似＋較正でいたい」、屋内測位はしない方針、iOSサイレントONは無音（端末仕様）。

主要数値・早見表（Q&Aの裏付け）

項目	値	参照（ファイル / 定数・関数）
使用AIモデル	<code>gemini-2.5-flash</code>	<code>api/gemini.js</code>
temperature（クイズ）	0.3	<code>api/gemini.js</code>
temperature（巡回順・既定）	0.7	<code>api/gemini.js</code>
AI呼び出しタイムアウト	8秒	<code>gemini_service.dart</code> <code>_timeout</code>
巡回順“考え中”演出	最低1.2秒	<code>shopping_list_screen.dart</code>
走行検知しきい値	4.0 m/s² （重力除外の合成加速度）	<code>motion_service.dart</code> <code>_walkAlertThreshold</code>
走行と判定する継続時間	0.5秒 （500ms）	<code>motion_service.dart</code> <code>_sustainMs</code>
走行ロック解除レイ	1200ms	<code>motion_service.dart</code> <code>calmTimer</code>
加速度の平滑化	EMA <code>ema*0.7 + mag*0.3</code>	<code>motion_service.dart</code>
方位スムージング係数	EMA 0.2	<code>compass_service.dart</code> <code>_smoothing</code>
方位の最小変化（チラつき防止）	1.0度	<code>compass_service.dart</code> <code>_minDeltaDeg</code>
ポイント→スタンプ	2ポイント=1スタンプ	<code>point_service.dart</code> <code>pointsPerStamp</code>
シール交換しきい値	5スタンプ=10ポイント	<code>point_service.dart</code> <code>stampsToRedeem</code> / <code>stickerThreshold</code>
1クイズ正解で得るポイント	1ポイント （= 正解数が獲得ポイント）	<code>navigation_screen.dart</code> / <code>sticker_screen.dart</code>
商品データ	8品（固定）	<code>models.dart</code> <code>sampleItems</code>
売り場マスタ	25区画（pathIndex 0～24・x,y座標）	<code>models.dart</code> <code>storeAreas</code>

項目	値	参照（ファイル / 定数・関数）
難易度	3段階（未就学/小学生/高学年・既定=小学生）	<code>level_service.dart</code> <code>levels</code> / <code>defaultId</code>

Q&A 本編

1. アーキテクチャ / 設計全般

Q1-1. 全体構成と層分けは？

回答：「Flutter Web の単一SPAです。UIの `screens/`、ロジックの `services/`、固定データの `models.dart` に分け、AIだけはサーバー側（Vercel Function）に逃がしています。フロントは同一オリジンの `/api/gemini` を叩くだけなので、APIキーもフロントに出ません。」 **根拠：** `main.dart`（`MaterialApp` → `HomeScreen` 起点）／ `screens/`（8画面）／ `services/`（`gemini`・`level`・`point`・`compass`・`motion`・`speech`）／ `models.dart`（`ShoppingItem`・`StoreArea`）／ `api/gemini.js`。 **深掘り：** 状態は画面ローカルの `StatefulWidget` + `setState`。横断状態は累計ポイントと難易度だけで、`shared_preferences` に保存。重い状態管理ライブラリは入れていません（`pubspec.yaml` に `Provider/Riverpod` なし）。

Q1-2. なぜ Flutter Web なのか？

回答：「インストール不要でURL一発。親のスマホでも店舗の貸出端末でも開けて、iOS/Android を1コードで両対応できます。“その場で実機デモ”に最適だからです。」 **根拠：** カメラは `mobile_scanner`、方位・加速度は Web の `DeviceOrientation/DeviceMotion` を直接利用（`compass_service.dart`／`motion_service.dart`）。 **深掘り：** センサー系は `dart:html` 依存の Web 専用実装で、ネイティブ化時は `sensors_plus` 等へ差し替える前提（各ファイル冒頭コメント）。公開API（`headingStream`／`requestPermission`）を保ったまま実装だけ差し替えられます。

Q1-3. なぜ大きな状態管理ライブラリを使わない？

回答：「画面ごとに状態が閉じていて、共有が要るのは累計ポイントと難易度だけ。これは端末ローカル保存で足ります。規模に対して過剰投資はしない判断です。」 **根拠：** `point_service.dart`（キー `total_points`）と `level_service.dart`（キー `quiz_level` + 同期キャッシュ `currentId`）が唯一の横断状態。 **深掘り：** 難易度は `await` できない箇所で参照するため、`LevelService.currentId` の同期キャッシュで即取得できるようにしています。

2. AI (Gemini)

Q2-1. どのモデルを、どう呼んでいる？

回答：「`gemini-2.5-flash` を Vercel のサーバー関数経由で呼びます。フロントは直接 Google を叩きません。レスポンスは JSON 固定にしています。」 **根拠：** `api/gemini.js` (`gemini-2.5-flash:generateContent` 、 `responseMimeType: application/json`) / エンドポイントは `/api/gemini` (`gemini_service.dart`)。

Q2-2. AIはいつ・何回呼ぶ？（コスト抑制）

回答：「呼ぶのは2か所だけ。買い物開始時に“巡回順”を1回、各商品の到着時に“クイズ”を1回。ナビ中に毎フレーム呼ぶようなことはしません。」 **根拠：** 巡回順 = `shopping_list_screen.dart` (`GeminiService.suggestVisitOrder`)、クイズ = `quiz_screen.dart` (`GeminiService.generateQuiz`)。 **深掘り：** 方位・走行検知・進行・スキャンは全てローカルで API 不要。1回のおつかいの呼び出しは「**1 + 商品数**」回に収まり、コストが予測可能です。

Q2-3. temperature とプロンプト設計は？

回答：「クイズは事実ベースなので temperature 0.3 に下げて創作の暴走を抑えます（巡回順は既定0.7）。プロンプトで“与えた事実だけを根拠に作れ／4択中ちょうど1つだけ正解／こわい・差別はNG”と縛っています。」 **根拠：** `api/gemini.js` (quizブロックで `temperature=0.3`、ルール文)。年齢別の言葉づかいは `levelHint` をプロンプトに差し込み（指示文は `level_service.dart` の各 `hint`)。

Q2-4. 入出力JSONの形は？

回答：「巡回順は `{order:[id,...]}`、クイズは `{question, choices[4], correctIndex, explanation}` の厳密スキーマです。」 **根拠：** 出力指示は `api/gemini.js`、受信パースは `gemini_service.dart`。 **深掘り：** 入力に子どもの個人情報を含めません。巡回順は `id/name/areaId`、クイズは商品名・売場・事実文のみ送信 (`gemini_service.dart`)。

Q2-5. レイテンシ（遅さ）対策は？

回答：「タイムアウト8秒。超えたら固定データへ即フォールバックして UI は止めません。巡回順では“考え中”演出を最低1.2秒見せて体感も整えています。」 **根拠：** `gemini_service.dart` (`_timeout=8秒`、`.timeout()`) / 演出は `shopping_list_screen.dart`。 **深掘り：** クイズ生成はスキャン後の「カゴに追加」演出 (`_Phase.adding`) の裏で走らせるので、待ち時間が体感に出にくい設計です (`quiz_screen.dart`)。

Q2-6【追加】. Geminiが壊れたJSONや余計な文を返したら？

回答：「二重で守ります。サーバー側で JSON 出力を強制し、パースに失敗したら 502 を返す。フロントは受け取った後に型・件数・重複・index・空文字を再検証し、少しでも崩れたら固定クイズに落とします。」 **根拠：**サーバー = `api/gemini.js` (`responseMimeType` 強制 + `JSON.parse` 失敗で502) / フロント = `gemini_service.dart` (`generateQuiz` 内の検証)。

3. ハルシネーション対策

Q3-1. AIが事実を“発明”しない仕組みは？

回答：「4層の多層防御です。①手書きの“正しい事実”だけを根拠に出題（グラウンディング）②temp0.3 + 禁止ルールの厳格プロンプト③フロントで多段検証④全部ダメなら手書き固定クイズへフォールバック。だから誤クイズは画面に出ません。」 **根拠：**① `gemini_service.dart` (`fact: item.explanation` を渡す) + `api/gemini.js` ② `api/gemini.js` ③ `gemini_service.dart` (検証ブロック) ④ `quiz_screen.dart` (`_quiz?.x ?? widget.item.x` で null時に固定値)。 `docs/引き継ぎ.md` に4層を明記。

Q3-2. 具体的にどんな検証を書いている？（クイズ）

回答：「選択肢がちょうど4個か、空文字がないか、4つすべて異なるか（重複NG）、`correctIndex` が0～3の整数か、問題文・解説が空でないか。1つでも崩れたら丸ごと捨てて固定クイズに落とします。」 **根拠：** `gemini_service.dart` の `generateQuiz` 内： `choices.length != 4` / 空選択肢NG / `toSet().length != 4` (重複) / `correctIndex` 範囲 / 質問・解説の空チェック。サーバー側も「4つ・重複なし」を指示 (`api/gemini.js`) で二重担保。

Q3-3. 巡回順の検証（回遊性ガード）は？

回答：「AIが返した id が入力と過不足なく一致しなければ採用しません。さらに“後戻り回数”を数え、AI順がローカル順より行ったり来たりが多ければローカル順を優先します。」 **根拠：** id一致 = `gemini_service.dart` (`suggestVisitOrder`、集合不一致なら null) / 回遊性ガード = `shopping_list_screen.dart` (`_backtrackCount` と採用条件)。ローカル基本順は売り場 `pathIndex` 昇順（入口→レジの一方向スイープ）なので、フォールバックでも必ず後戻りしません。

4. セキュリティ / プライバシー

Q4-1. APIキーはどう守っている？

回答：「鍵はフロントに一切置きません。Vercel の環境変数 `GEMINI_API_KEY` に入れ、サーバー関数の中だけで使います。フロントは `/api/gemini` を叩くだけで鍵を知りません。」 **根拠：** `api/gemini.js` (`process.env.GEMINI_API_KEY`、未設定なら500、サーバーでURLに付与)。クライアントのビルド成果物に鍵が混入する経路がありません。

Q4-2. CORS とプロキシ設計は？

回答：「Vercel 関数がプロキシ兼 CORS ハンドラです。OPTIONS プリフライトに200、POSTのみ受け付け。Google API はサーバー間通信なのでブラウザの CORS 制約も回避できます。」

根拠： `api/gemini.js` (`Access-Control-Allow-*` / OPTIONS 200 / POST以外405)。SPA ルーティングは `/api/` を除外 (`vercel.json` の `rewrites` 正規表現 `(?!api/)`)。 **深掘り：** 現状 `Allow-Origin: *` はデモ簡便さのため。本番では自ドメインに絞れます。

Q4-3. 子どもの位置情報やカメラ映像をサーバーに送っていない？

回答：「送っていません。方位・加速度は端末内で処理し外に出しません。カメラ映像も端末内で読み取るだけ。AIに送るのは商品名・売場名・事実文だけです。GPSも使いません。」 **根拠：** センサーは `dart:html` で端末内処理（外部送信コードなし）。バーコードは `mobile_scanner` で検出し `rawValue` を画面遷移に使うのみ (`quiz_screen.dart`)。AI送信ペイロードに位置・画像なし (`gemini_service.dart`)。屋内測位はしない方針 (`docs/引き継ぎ.md`)。

5. センサー / 方位

Q5-1. 位置情報を使わずどうやって方角を出す？【重要】

回答：「売り場ごとに固定した座標 (x,y) を持っていて、“1つ前の売り場→次の売り場”の方向を `atan2` で計算し、端末の方位を引いて針の角度にしています。GPSや屋内測位は使いません。」 **根拠：** 方位角計算 = `navigation_screen.dart` (座標差から `atan2(dx,dy)`)。針角度への変換 = `compass_screen.dart`。座標は `models.dart` の `storeAreas` (x: 左→右 0..100、y: 下→上 0..100)。

Q5-2. 方位センサーの取り方と「絶対/相対の非混在」は？

回答：「ブラウザの DeviceOrientation を購読し、北=0の0～360度に正規化します。Android は絶対方位イベントの `alpha`、iOS は `webkitCompassHeading`。絶対方位が一度でも取れたら相対

方位は捨てます。 混ぜると向きによって象限ごとにズレるからです。」 **根拠：**

`compass_service.dart` (`headingStream`、`gotAbsolute` フラグで相対を無視)。

Q5-3. 針のブレ対策（円環スムージング）は？

回答：「角度を単位ベクトル(cos,sin)に直して EMA で平滑化します。これなら359度と1度の境界をまたいでも正しく平均できます（単純平均だと180度になってしまう）。1度未満の変化は捨ててチラつきを止めます。」 **根拠：** `compass_service.dart` (`sx/sy` のEMA=係数 `_smoothing=0.2`、`_minDeltaDeg=1.0`、`atan2` で角度復元)。針アニメは累積回転で最短回り (`compass_screen.dart`)。

Q5-4. 店の北とマップの北のズレはどう吸収する？（校正の正当性）【追加】

回答：「入店時にワンタップ校正します。“いまの前方を次の目的地方向とみなす”ことで、マップ北と地磁気北のズレ（オフセット）を1回で合わせます。この補正值は static で全ミッション保持するので毎回やり直す必要はありません。」 **根拠：** `compass_screen.dart` (`_calibrateToFront` : `storeNorthOffsetDeg = heading - targetBearing`)、`compass_service.dart` (`storeNorthOffsetDeg` は static)。

Q5-5. iOS/Android差と権限は？

回答：「iOS はセンサー許可をユーザーのタップ起点でしか出せないなので、ボタンを押させてから要求します。Android/PC は権限不要で即購読。3秒値が来なければタップ案内に切り替えます。」 **根拠：** `compass_service.dart` (`requestPermission` : iOS の `DeviceOrientationEvent.requestPermission()` のみ Promise 処理、無ければ true) / `compass_screen.dart` (タップ起点配線・3秒タイマー)。例外時も true で続行し止めません。

Q5-6. 精度の限界は？（正直に）

回答：「方角は“だいたい”です。屋内測位はせず、店内マップを近似座標に落として方位角を計算し、ワンタップ校正でマップ北を合わせて実用精度に寄せています。地磁気センサー自体の屋内誤差は端末依存で、そこは割り切っています。」 **根拠：** 近似座標 (`models.dart`) + 校正 (`compass_screen.dart`)。 `docs/引き継ぎ.md` 「方角は近似座標+校正で“だいたい”」。

6. 安全機能

Q6-1. 走行検知のしきい値は？

回答：「重力を除いた合成加速度が **4.0 m/s²** を超え、それが **0.5秒** 続いたら“走り”と判定します。歩行1~3、早歩き3~5、走り6~15くらいなので、ナビ用のゆっくり歩きは許して、それよ

り速いと警告します。」 **根拠：** `motion_service.dart` (`_walkAlertThreshold=4.0` / `_sustainMs=500` / 合成加速度 `sqrt(x2+y2+z2)` / 重力除外値が無い端末は約9.8を引いて近似)。

Q6-2. 単発の衝撃で誤発火しない？

回答：「しません。瞬間のスパイクは無視します。まずEMAで平滑化し、さらに“0.5秒継続”を条件に。解除も1.2秒の余裕を持たせてチラつきを防ぎます。状態が変わったときだけ通知します。」 **根拠：** `motion_service.dart` (EMA `ema*0.7+mag*0.3` / `overSince` の継続判定 / 1200ms の `calmTimer` / 状態変化時のみ stream 送出)。センサー非対応・未許可端末では何も流さずデモは進行。

Q6-3. 危険アラートと走行ロックの中身は？

回答：「走行検知中は赤い全画面オーバーレイで“とまって！”を被せ、音声でも止めるよう促します。さらに2件目のミッションで“すべりやすい”等の危険ダイアログを音声つきで一度だけ出します。危険判定はAIではなく固定の文言・ロジックです。」 **根拠：** 走行ロック = `compass_screen.dart` (`_RunLockOverlay` + 音声) / 危険アラート = `navigation_screen.dart` (2件目で1回、 `hazardAlerts[0]` + 音声) / 常時バー `_SlowWalkBar` / お約束画面 `safety_pledge_screen.dart` / 危険文言は `models.dart` `hazardAlerts` 。

7. データ / スケーラビリティ

Q7-1. 今のデータは固定？本番はどうする？

回答：「デモは `models.dart` に8品+25売り場を直書きしています。本番は商品マスタ・クイズDB・JANコード連携に差し替える設計で、モデルにはすでに `janCode` を持たせています。商品名は架空ブランド＋一般名なので、実在商品に依存せずデータ差し替えだけで本番化できます。」 **根拠：** `models.dart` (`sampleItems` 8品 / `storeAreas` 25区画 / `janCode` フィールド / 差し替え前提コメント)。

Q7-2. 25売り場の座標と巡回順の根拠は？

回答：「店内マップを開発時にデジタル化し、25区画に“入口→レジの一方向スワイプ順 (pathIndex)”と近似座標を与えています。巡回順はこの並びが基準で、座標は次売場の方角計算に使います。」 **根拠：** `models.dart` `storeAreas` (pathIndex 0～24、x,y)。ローカル順は pathIndex 昇順 (`shopping_list_screen.dart`)、方角は `navigation_screen.dart`。座標精緻化は今後の改善項目 (`docs/引き継ぎ.md`)。

Q7-3. 難易度3段階の中身は？

回答：「未就学・小学生(1～3年)・高学年(4～6年)の3段階、既定は小学生。レベルごとに言葉づかいの指示文をAIに渡し、ひらがな量や漢字・問い方を変えます。高学年は常用漢字を使い“理由・仕組みを考えさせる問い方”に。どのレベルでも“事実外を足さない”制約は共通です。」 **根**

拠： `level_service.dart` (`levels` の各 `hint`、既定 `defaultId=2`)。選択UIは `home_screen.dart`、クイズ生成へ反映は `quiz_screen.dart` → `api/gemini.js`。

Q7-4. スケールするのか？

回答：「アプリ層は静的SPA+サーバーレス関数なので Vercel で水平スケールします。データ層は商品マスタ/クイズDB/JAN連携に差し替える前提で、固定データを置き換えるだけで店舗展開できます。ボトルネックは Gemini のレート上限ですが、呼び出しが“開始時+商品ごと”に限定されるため見積もりやすく、固定フォールバックで上限到達時も止まりません。」 **根拠：** サ

ーバーレス `api/gemini.js` + Vercel 配信 (`vercel.json`)、差し替え前提のデータ層 (`models.dart`)。

8. デプロイ / インフラ

Q8-1. ビルドとデプロイの仕組みは？

回答：「Vercel の buildCommand で Flutter 本体を clone して `flutter build web --release` し、 `build/web` を配信します。 `main` に push すれば自動デプロイ。実質これが CI/CD です。」

根拠： `vercel.json` (buildCommand / `outputDirectory: build/web`)、 `docs/引き継ぎ.md` (main push → Vercel 自動)。

Q8-2. SPAのルーティング (rewrites) は？

回答：「 `/api/` 以外のパスは全部 `index.html` に書き換えて Flutter 側ルーティングへ渡します。API だけ素通し。これでディープリンクや更新時の404を防ぎます。」 **根拠：** `vercel.json` (`source: /(?!api/).*` → `/index.html`)。

Q8-3. ビルド成果物をGit管理外にする理由は？

回答：「 `build/` は Vercel が毎回ソースから生成するので、コミットすると差分が荒れて衝突の温床になるだけ。ソース (`lib/` ・ `api/` ・ `vercel.json`) を唯一の真実にして、成果物は決定的に再生成します。」 **根拠：** `.gitignore` (`/build/`)、 `vercel.json` (毎ビルドで生成)。

9. 既知の限界（正直に答える）

Q9-1. iOSで音声が届らないことがある？

回答：「あります。iOS Safari は音声をユーザータップ起点でしか鳴らせないので、お約束画面のスタートで無音発話して“解放”しています。ただし iPhone のサイレントスイッチ ON だけは端末仕様で無音です。ここは正直に限界です。非対応ブラウザでも無音で続行し、デモは止めません。」 **根拠：** `speech_service.dart`（`unlock` / 日本語 voice 選択 / 例外を握って無音続行） + 呼び出しは `safety_pledge_screen.dart`。`docs/既知の課題.md`・`docs/引き継ぎ.md`。

Q9-2. 屋内測位はしないのか？

回答：「しません。GPSは屋内で弱く、専用ビーコンはコストが見合わない。近似座標 + ワンタップ校正で“だいたいの方角”に割り切っています。将来は“マップ基準モード（肉=上・野菜=右を固定）”への切替案があります。」 **根拠：** `docs/引き継ぎ.md`（屋内測位はしない方針・将来のAモード案）、校正は `compass_screen.dart`。

Q9-3. `flutter analyze` のエラーは大丈夫？

回答：「`dart:js_util` 周りで解析エラーが2件出ますが、静的解析の偽陽性で実ビルド・実行には影響しません。Web専用センサー実装の宿命で、ネイティブ化で `sensors_plus` に差し替えれば消えます。」 **根拠：** `docs/引き継ぎ.md`（「error 2件は `dart:js_util`（compass/motion）の解析偽陽性＝無視可」）。

10. 鋭いツッコミ想定

Q10-1. AIが間違えたら？

回答：「間違ったクイズは表示されません。フロントの多段検証で1つでも崩れたら丸ごと捨てて手書きの固定クイズに差し替えます。AIは“あれば使う、ダメなら無かったことにする”扱いです。」 **根拠：** `gemini_service.dart`（検証 + null）、`quiz_screen.dart`（フォールバック）。

Q10-2. ネットが切れたら？

回答：「全部ローカルで完走します。AI呼び出しは8秒タイムアウトで null、巡回順はローカル基本順、クイズは固定クイズに落ちます。ネットの状態に関係なくデモは最後まで動きます。」 **根拠：** `gemini_service.dart`（タイムアウト・例外で null）、`shopping_list_screen.dart`（巡回順フォールバック）、`docs/引き継ぎ.md`（一気通貫で動く）。

Q10-3. なぜ“最短経路”を出さないのか？

回答：「物理ルートは出しません。出すのは“回る順番（巡回順）”だけです。理由は2つ。屋内の正確な現在地が取れないので経路は責任を持ってない。そして実店舗は一方向に回るのが自然なので、後戻りの少ない順番のほうが実用的だからです。」 **根拠：** AIは巡回順のみ

（ `gemini_service.dart` コメント「物理ルートは生成しない」、 `api/gemini.js` ）。ローカルは `pathIndex` 一方向スイープ+回遊性ガード（ `shopping_list_screen.dart` ）。 **深掘り：** 「最短」という言葉自体使いません。正しくは「**後戻りの少ない巡回順**」です。

Q10-4. なぜ専用のセンサーパッケージを使わない？

回答：「Flutter Web だからです。 `sensors_plus` 等はネイティブ前提で、Web では標準の `DeviceOrientation/DeviceMotion` を直接使うのが確実、依存も減らせます。ネイティブ化時は同じ公開APIのまま1ファイルだけ差し替える前提で設計しています。」 **根拠：**

`compass_service.dart` / `motion_service.dart` の冒頭コメント（差し替え方針）と公開API（ `headingStream` / `requestPermission` ）。

Q10-5. コストは？

回答：「1回のおつかいで“巡回順1回+商品数ぶんのクイズ”だけ。ナビ中は呼びません。

`gemini-2.5-flash` は安価で出力も短いJSON1問。予測可能で低コストです。フォールバックがあるので、コスト上限でAIをオフにしてもアプリは成立します。」 **根拠：** 呼び出しは2か所のみ（ `shopping_list_screen.dart` / `quiz_screen.dart` ）、モデルは `api/gemini.js` 。

Q10-6【追加】. 連打やリトライでポイントは二重加算されない？（冪等性）

回答：「されません。獲得ポイントの累計加算は完了画面の初期化で1回だけ。スキャンは多重発火防止フラグで1回だけ検出。危険アラートも1回だけ出すようフラグ管理しています。」 **根**

拠： `sticker_screen.dart` （ `initState` で `PointService.add` を1回）、 `quiz_screen.dart` （ `_handled` フラグ）、 `navigation_screen.dart` （ `_hazardShown` フラグ）。

Q10-7【追加】. 途中追加（リスト外スキャン）商品のクイズはどう事実保証する？

回答：「リスト外商品は商品名が取れないので、商品名に依存しない“地産地消の汎用クイズ”+固定の正しい事実を使います。JAN→商品DB連携が今回権限外という前提を踏まえた割り切りです。」 **根拠：** `models.dart` `bonusItem` （ `areaId: 'unknown'` 、固定 `explanation` を根拠に出題）。

補足：ポイント体感速度（先回り回答）

「1クイズ正解=1ポイント、2ポイントで1スタンプ、5スタンプ（10ポイント）でシール交換」なので、**5商品を全問正解しても1回のおつかいでは約2スタンプ（4ポイント）**。シール交換は数回の来店で到達する設計です。「交換まで遠くないか」と聞かれたら、**継続来店の動機づけ（リテンション）を意図した設計**だと説明できます（しきい値は `point_service.dart` の定数変更だけで調整可能）。

相互チェック記録

- **作成エージェント**：実コードを読み、コード根拠つきで30問超を起草。
- **検証エージェント**：全主張を実コードと突き合わせて事実確認。判定は「**技術精度 高 (A) ・捏造や方針違反ゼロ**」。行番号の微ズレを補正し、重要質問5問を追加。
- 本資料は上記2エージェントの相互チェックを反映した最終版。参照は**関数名 ・ 定数名中心**にしてある（コード編集で行番号がずれても陳腐化しにくくするため）。